

Template – Exception – Composite

Lập trình hướng đối tượng

Hồ Tuấn Thanh

htthanh@fit.hcmus.edu.vn

Nội dung

- ☐ Function template
- ☐ Class template
- ☐ Xử lí ngoại lệ
- ☐ Mẫu Composite

Function template

Ví dụ cơ bản

```
template<class T>  
T findMax(T x, T y);
```

```
template<class T>  
T findMax(T x, T y)  
{  
    if(x>y)  
        return x;  
    return y;  
}
```

```
void main()  
{  
    int a1=10;  
    int b1=20;  
    cout<<findMax(a1,b1)<<endl;  
  
    double a2=10.1;  
    double b2=-2.3;  
    cout<<findMax(a2,b2)<<endl;  
}
```

Vấn đề 1: Cài đặt nhiều file

□ Func.h

```
#include<iostream>
using namespace std;

template<class T>
T findMax(T x, T y);
```

□ Func.cpp

```
#include "Func.h"

template<class T>
T findMax(T x, T y)
{
    if(x>y)
        return x;
    return y;
}
```

Vấn đề 1: Cài đặt nhiều file

❑ main.cpp

```
#include "Func.h"

void main()
{
    int a1=10;
    int b1=20;
    cout<<findMax(a1,b1)<<endl;

    double a2=10.1;
    double b2=-2.3;
    cout<<findMax(a2,b2)<<endl;
}
```

❑ Lỗi

Error List	
3 Errors 0 Warnings 0 Messages	
	Description
1	error LNK2019: unresolved external symbol "double __cdecl findMax<double>(double,double)" (??\$findMax@N@@@YANNN@Z) referenced in function _main
2	error LNK2019: unresolved external symbol "int __cdecl findMax<int>(int,int)" (??\$findMax@H@@@YAHHH@Z) referenced in function _main
3	fatal error LNK1120: 2 unresolved externals

Vấn đề 1: Cài đặt nhiều file

❑ Sửa lỗi cách 1.

```
#include "Func.cpp"

void main()
{
    int a1=10;
    int b1=20;
    cout<<findMax(a1,b1)<<endl;

    double a2=10.1;
    double b2=-2.3;
    cout<<findMax(a2,b2)<<endl;
}
```

```
template
int findMax(int x, int y);

template
double findMax(double x, double y);
```

❑ Sửa lỗi cách 2.

- (Thêm khai báo vào cuối file Func.cpp)

Vấn đề 2: Kiểu dữ liệu tự định nghĩa

```
#pragma once
L
class Fraction
{
private:
    int nu;
    int de;
public:
    Fraction(void);
    ~Fraction(void);
};
```

```
template<class T>
T findMax(T x, T y)
{
    if (x > y)
        return x;
    return y;
}
```

```
void main()
{
    int a1=10;
    int b1=20;
    cout<<findMax(a1,b1)<<endl;

    double a2=10.1;
    double b2=-2.3;
    cout<<findMax(a2,b2)<<endl;

    Fraction f1;
    Fraction f2;
    cout<<findMax(f1,f2)<<endl;
}
```


Vấn đề 2: Kiểu dữ liệu tự định nghĩa

```
int operator>(Fraction f);  
friend ostream& operator<<(ostream& os, Fraction f);
```

```
int Fraction::operator>(Fraction f)  
{  
    int delta=nu*f.de-de*f.nu;  
    if(delta>0)  
        return 1;  
    return 0;  
}  
ostream& operator<<(ostream& os, Fraction f)  
{  
    os<<f.nu<<"/"<<f.de;  
    return os;  
}
```

```
template  
Fraction findMax(Fraction x, Fraction y);
```

```
void main()  
{  
    int a1=10;  
    int b1=20;  
    cout<<findMax(a1,b1)<<endl;  
  
    double a2=10.1;  
    double b2=-2.3;  
    cout<<findMax(a2,b2)<<endl;  
  
    Fraction f1;  
    Fraction f2;  
    cout<<findMax(f1,f2)<<endl;  
}
```

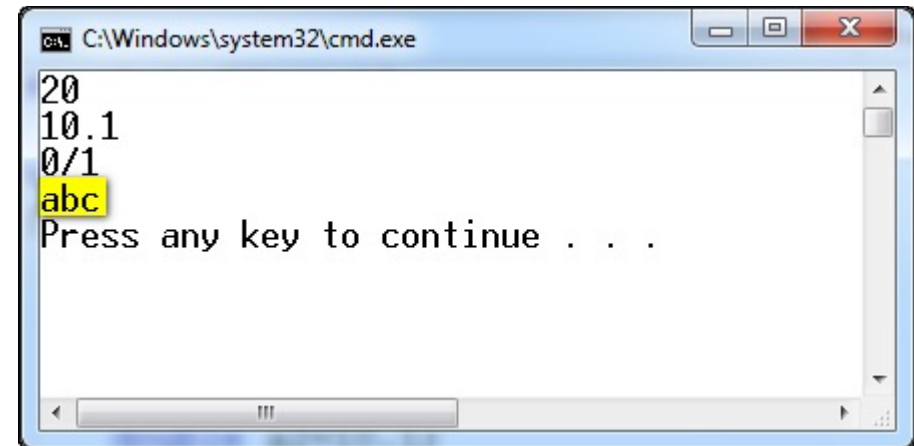
Vấn đề 3: Trường hợp đặc biệt

```
void main()
{
    int a1=10;|
    int b1=20;
    cout<<findMax(a1,b1)<<endl;

    double a2=10.1;
    double b2=-2.3;
    cout<<findMax(a2,b2)<<endl;

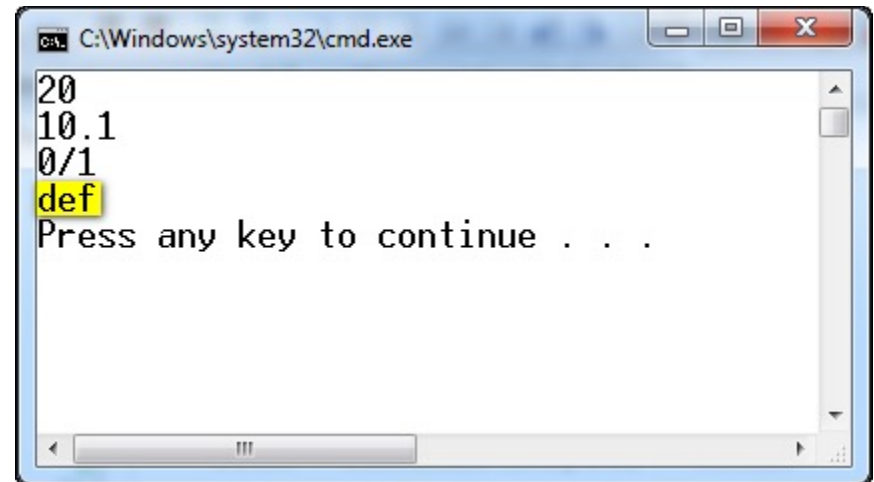
    Fraction f1;
    Fraction f2;
    cout<<findMax(f1,f2)<<endl;

    char str1[]="abc";
    char str2[]="def";
    cout<<findMax(str1,str2)<<endl;
}
```



Vấn đề 3: Trường hợp đặc biệt

```
template <>
char* findMax(char *x, char * y)
{
    int res=stricmp(x,y);
    if(res>0)
        return x;
    return y;
}
```



A screenshot of a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The window displays the output of the findMax function: "20", "10.1", and "0/1". The word "def" is highlighted in yellow. Below the output, the text "Press any key to continue . . ." is displayed.

Class template

Ví dụ

```
#pragma once
#include<iostream>
using namespace std;

template<class T>
class TwoEleClass
{
private:
    T x;
    T y;
public:
    TwoEleClass(void);
    ~TwoEleClass(void);
    TwoEleClass (T _x, T _y);
    T findMax();
};
```

```
void main()
{
    TwoEleClass<int> t(5, 6);
    cout<<t.findMax()<<endl;
}
```

```
#include "TwoEleClass.h"

template<class T>
TwoEleClass<T>::TwoEleClass(void)
{
}

template<class T>
TwoEleClass<T>::~~TwoEleClass(void)
{
}

template<class T>
TwoEleClass<T>::TwoEleClass (T _x, T _y)
{
    x=_x;
    y=_y;
}

template<class T>
T TwoEleClass<T>::findMax()
{
    if(x>y)
        return x;
    return y;
}

template class TwoEleClass<int>;
```

Xử lý ngoại lệ

Khi có lỗi lúc chạy chương trình

- ❑ Thông báo lỗi và kết thúc chương trình
- ❑ Trả về giá trị đặc biệt thể hiện chương trình có lỗi
- ❑ Sử dụng biến toàn cục lưu trạng thái lỗi
- ❑ Gọi một hàm qui ước khi có lỗi

Lợi ích của xử lý ngoại lệ

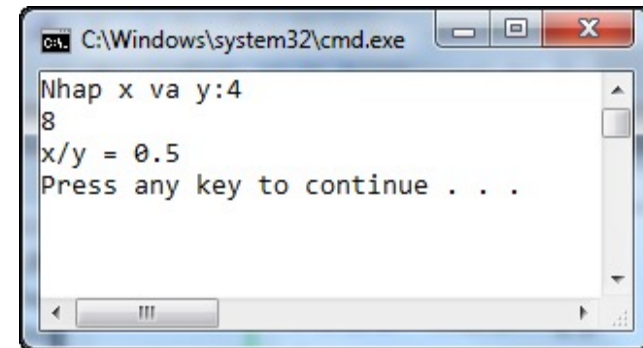
- ❑ Tiếng Anh: **exception handling**
- ❑ Tách biệt phần xử lý lỗi ra đoạn chương trình bình thường
- ❑ Thông báo lỗi ra bên ngoài cho các hàm mức cao hơn xử lý

try, throw và catch

- ❑ Cơ chế xử lý ngoại lệ trong C++/C#/Java được xây dựng với 3 từ khóa chính: try, throw và catch.
- ❑ Khối lệnh try chứa đoạn code mà ta cho rằng sẽ có trường hợp phát sinh ngoại lệ, cần được theo dõi.
- ❑ Lệnh throw dùng để tạo ra một ngoại lệ.
 - Lệnh throw có thể do máy tính sinh ra.
 - Hoặc do người lập trình throw khi cần.
- ❑ Khối lệnh catch chứa đoạn code xử lý khi ngoại lệ đã xảy ra trong khối lệnh try.

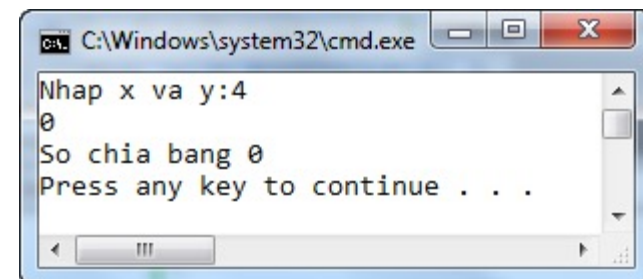
Ví dụ

```
void main()
{
    int x;
    int y;
    try
    {
        cout<<"Nhập x và y:";
        cin>>x>>y;
        if(y==0)
            throw 0;
        cout<<"x/y = "<<1.0*x/y<<endl;
    }
    catch(int i)
    {
        cout<<"Số chia bằng 0"<<endl;
    }
}
```



C:\Windows\system32\cmd.exe

```
Nhập x và y:4
8
x/y = 0.5
Press any key to continue . . .
```



C:\Windows\system32\cmd.exe

```
Nhập x và y:4
0
Số chia bằng 0
Press any key to continue . . .
```

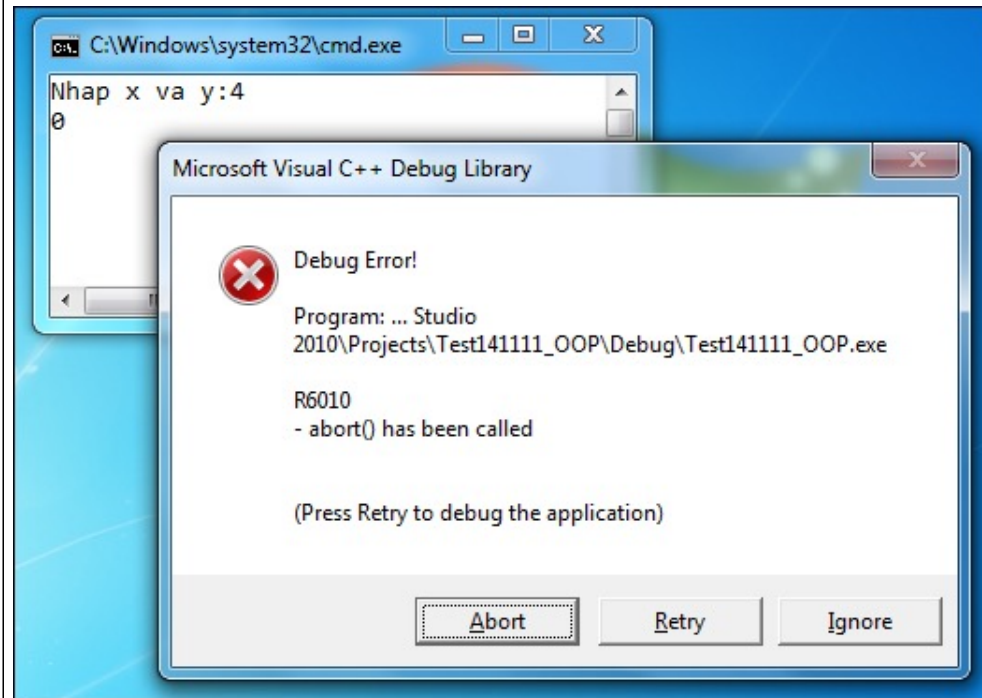
Ví dụ

- ❑ Với những trường hợp bình thường ($y \neq 0$), các câu lệnh trong khối try sẽ được chạy.
 - Câu lệnh tính thương ở sau throw sẽ được chạy.
- ❑ Với trường hợp y bằng 0, ngay sau lệnh throw, các câu lệnh trong khối catch sẽ được chạy.
 - Câu lệnh tính thương ở sau throw sẽ ko được chạy.

throw và try

- ❑ Lệnh throw “ném” ra đối tượng kiểu gì thì lệnh “catch” phải bắt đúng kiểu đó.

```
void main()
{
    int x;
    int y;
    try
    {
        cout<<"Nhập x và y:";
        cin>>x>>y;
        if(y==0)
            throw 3.14;
        cout<<"x/y = "<<1.0*x/y<<endl;
    }
    catch(int i)
    {
        cout<<"Số chia bằng 0"<<endl;
    }
}
```

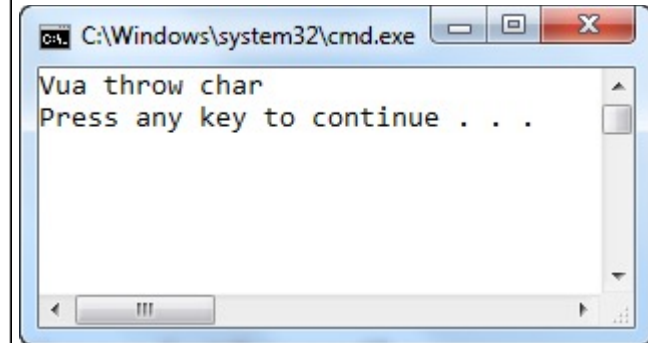


try và catch

- ❑ Một lệnh try có thể đi kèm với nhiều lệnh catch.
- ❑ Lệnh catch(...) dùng để bắt các ngoại lệ mà ko có lệnh catch nào xử lí được.
 - Thường là lệnh catch cuối cùng của một try.

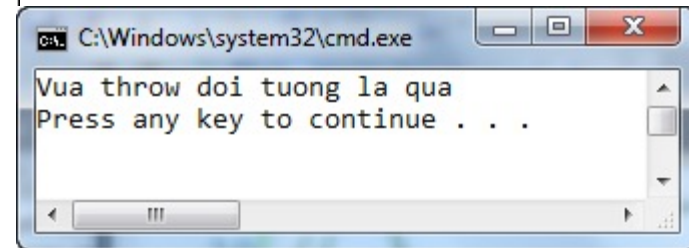
Ví dụ

```
void main()
{
    try
    {
        throw 'x';
    }
    catch(int i)
    {
        cout<<"Vua throw int"<<endl;
    }
    catch(double x)
    {
        cout<<"Vua throw double"<<endl;
    }
    catch(char c)
    {
        cout<<"Vua throw char"<<endl;
    }
    catch(...)
    {
        cout<<"Vua throw doi tuong la qua"<<endl;
    }
}
```



Ví dụ

```
void main()
{
    try
    {
        throw 'x';
    }
    catch(int i)
    {
        cout<<"Vua throw int"<<endl;
    }
    catch(double x)
    {
        cout<<"Vua throw double"<<endl;
    }
    catch(...)
    {
        cout<<"Vua throw doi tuong la qua"<<endl;
    }
}
```



try, throw và catch trong hàm

- Dĩ nhiên ta có thể try, throw và catch trong một hàm bất kì.

```
void f()
{
    try
    {
        throw 'x';
    }
    catch(int i)
    {
        cout<<"Vua throw int"<<endl;
    }
    catch(char c)
    {
        cout<<"Vua throw char"<<endl;
    }
}

void main()
{
    f();
}
```


throw trong, catch ngoài

- Dĩ nhiên ta có thể throw trong một hàm và để đoạn catch ở hàm cha (hàm mà gọi hàm chứa lệnh throw).
 - Đây là điều nên làm, vì thường khi có lỗi xảy ra, hàm hiện tại ko biết xử lí thế nào. Chỉ có hàm gọi hàm đó mới có đầy đủ thông tin để xử lí lỗi đó.

Ví dụ

```
void f()
{
    throw 'x';
}
void main()
{
    try
    {
        f();
    }
    catch(char c)
    {
        cout<<"Vua throw choi thoi, dung lo"<<endl;
    }
}
```

Loại đối tượng khi catch

- ❑ Thông thường ta ko catch các đối tượng có kiểu dữ liệu cơ sở (int, float, char,...) mà thường catch các đối tượng kiểu lớp.
 - Lí do, thông thường ta hay xây dựng lớp chứa thông tin lỗi.

Ví dụ

```
class MyException
{
private:
    string message;
public:
    MyException(string msg)
    {
        message=msg;
    }
    string getMessage()
    {
        return message;
    }
};
```

```
void main()
{
    int x;
    int y;
    try
    {
        cout<<"Nhap x va y:";
        cin>>x>>y;
        if(y==0)
            throw MyException("Divided by zero");
        cout<<"x/y = "<<1.0*x/y<<endl;
    }
    catch(MyException& ex)
    {
        cout<<ex.getMessage()<<endl;
    }
}
```

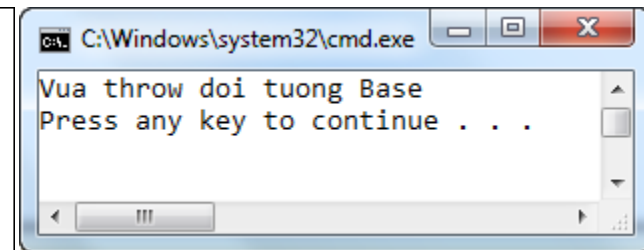
catch đối tượng lớp cơ sở

- ❑ Lệnh catch đối tượng lớp cơ sở, có thể bắt các lệnh throw đối tượng lớp dẫn xuất.
- ❑ Do đó, thường để lệnh catch lớp cơ sở ở dưới lệnh catch lớp dẫn xuất.
- ❑ Sau đây là một trường hợp sai.

Ví dụ

```
class Base{};  
class Derived: public Base{};
```

```
void main()  
{  
    try  
    {  
        Derived d;  
        throw d;  
    }  
    catch(Base& b)  
    {  
        cout<<"Vua throw doi tuong Base"<<endl;  
    }  
    catch(Derived& x)  
    {  
        cout<<"Vua throw doi tuong Derived"<<endl;  
    }  
}
```



Qui định các ngoại lệ của hàm

- ❑ Ta có thể qui định các ngoại lệ mà một hàm sẽ trả ra nếu có.
- ❑ Nếu trong hàm đó, ta “ném” ra một ngoại lệ ko được liệt kê thì hàm `unexpected()` sẽ được gọi.
 - Visual C++ ko ngăn một hàm “ném” ra một ngoại lệ mà chưa được liệt kê.

Ví dụ

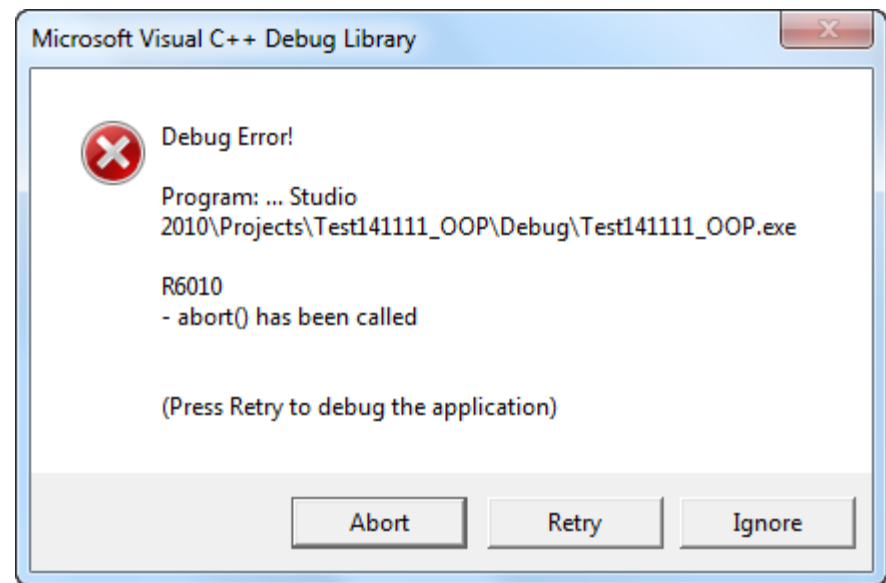
```
void f(int test) throw(int, char, double)
{
    if(test==1) throw test;
    if(test==2) throw 'x';
    if(test==3) throw 3.14;
}
```

```
void main()
{
    try
    {
        f(1);
    }
    catch(int n)
    {
        cout<<"Vua throw int = "<<n<<endl;
    }
    catch(char c)
    {
        cout<<"Vua throw char = "<<c<<endl;
    }
    catch(double x)
    {
        cout<<"Vua throw double = "<<x<<endl;
    }
}
```


Ví dụ

```
void f(int test) throw(int, char, double)
{
    if(test==1) throw test;
    if(test==2) throw 'x';
    if(test==3) throw 3.14;
    if(test==4) throw "abc";
}
```

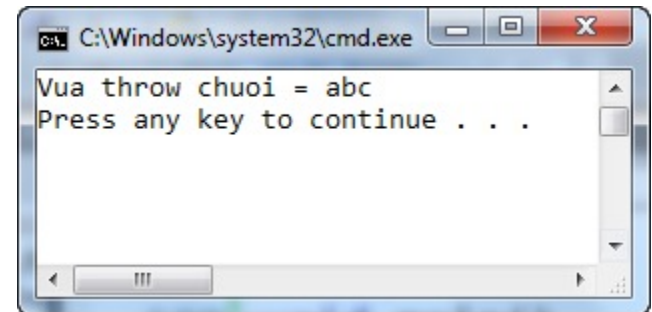
```
void main()
{
    try
    {
        f(4);
    }
    catch(int n)
    {
        cout<<"Vua throw int = "<<n<<endl;
    }
    catch(char c)
    {
        cout<<"Vua throw char = "<<c<<endl;
    }
    catch(double x)
    {
        cout<<"Vua throw double = "<<x<<endl;
    }
}
```



Ví dụ

```
void f(int test) throw(int, char, double)
{
    if(test==1) throw test;
    if(test==2) throw 'x';
    if(test==3) throw 3.14;
    if(test==4) throw "abc";
}
```

```
void main()
{
    try
    {
        f(4);
    }
    catch(char* s)
    {
        cout<<"Vua throw chuoi = "<<s<<endl;
    }
}
```



Ví dụ

- ❑ Nếu ta để danh sách rỗng thì có nghĩa rằng ta cam đoan hàm này ko có ngoại lệ.

```
void f(int test) throw()  
{  
    cout<<"Ham nay ko co ngoai le gi het!!!";  
}
```

Rethrow một ngoại lệ

- ❑ Tình huống - `h()` gọi `g()`, `g()` gọi `f()`:
 - Hàm `f` có ngoại lệ, throw lên cho `g()`; `g()` catch lại, xử lý xong, nhưng vẫn muốn throw lên cho `h()` biết.
 - Hàm `f` có ngoại lệ, throw lên cho `g()`; `g()` ko biết xử lý, lại throw lên cho `h()` xử lý.
- ❑ Có 2 cách rethrow:
 - `f()` throw gì cho `g()`, `g()` throw lại y chang cho `h()`
Dùng câu lệnh `throw`;
 - `f()` throw cho `g()`, `g()` tạo ngoại lệ mới rồi throw lại cho `h()`

Ví dụ

```
void f()
{
    throw 3.14;
}
```

```
void g()
{
    try
    {
        f();
    }
    catch(double x)
    {
        cout<<"Dang catch double o ham g"<<endl;
        throw;
    }
}
```

```
void main()
{
    try
    {
        g();
    }
    catch(double x)
    {
        cout<<"Dang catch double o ham main"<<endl;
    }
}
```

Ví dụ

```
void f()
{
    throw 3.14;
}
```

```
void g()
{
    try
    {
        f();
    }
    catch(double x)
    {
        cout<<"Dang catch double o ham g"<<endl;
        throw 'c';
    }
}
```

```
void main()
{
    try
    {
        g();
    }
    catch(char c)
    {
        cout<<"Dang catch char o ham main"<<endl;
    }
}
```

Rò rỉ bộ nhớ khi throw – Vấn đề

```
void test(int t, int a, int b)
{
    int *p;
    p=new int[t];
    // Blah blah blah
    if(a==b)
        throw "error";
    delete []p;
}
```

Rò rỉ bộ nhớ khi throw – Giải pháp 1

```
void test(int t, int a, int b)
{
    int *p;
    p=new int[t];
    // Blah blah blah
    if(a==b)
    {
        delete []p;
        throw "error";
    }
    delete []p;
}
```


Rò rỉ bộ nhớ khi throw – Giải pháp 2

- ❑ Với giải pháp 1, ta cần nhớ giải phóng vùng nhớ đã sử dụng trước khi “ném” lỗi.
- ❑ Một giải pháp tốt hơn là đưa các biến có sử dụng đến tài nguyên vào trong các lớp. Khi đó hàm hủy của lớp này sẽ đảm nhận vai trò giải phóng tài nguyên đang nắm giữ.
- ❑ Phương pháp này gọi là Resource Acquisition Is Initialization (RAII) do Bjarne Stroustrup đề nghị.

throw trong constructor

- ❑ OK, cứ throw bình thường.
- ❑ Nhớ hủy vùng nhớ đã cấp phát cho con trỏ nếu có

throw trong destructor

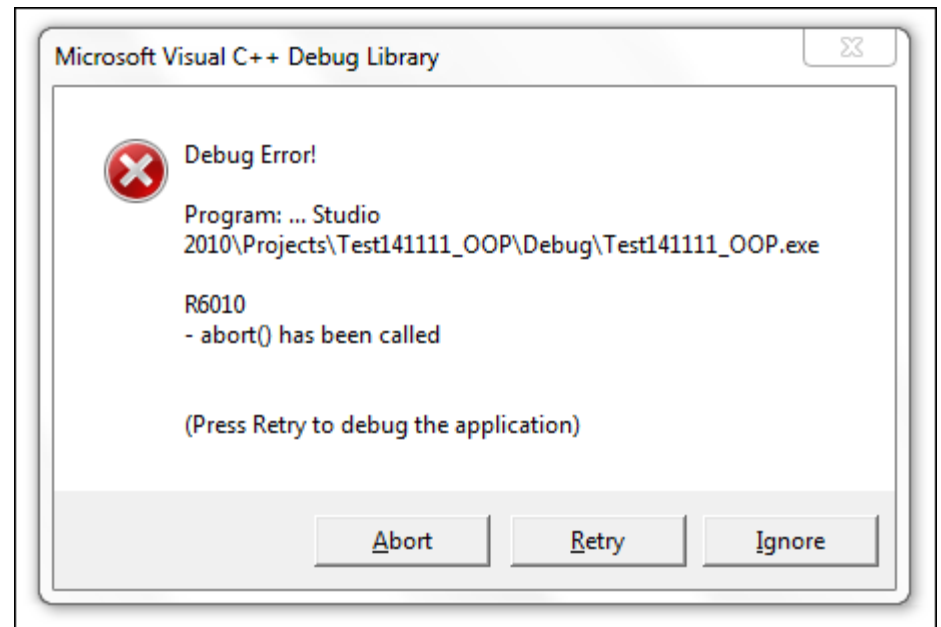
- ❑ Không nên “ném” ngoại lệ trong hàm hủy, mà chỉ ghi nhận lại lỗi (chẳng hạn dùng logger, ghi lỗi xuống file).

Ví dụ

```
class A
{
public:
    ~A()
    {
        throw 3.14;
    }
};
```

Ví dụ - trường hợp 1

```
void main()  
{  
    A a;  
    // Blah blah blah  
}
```



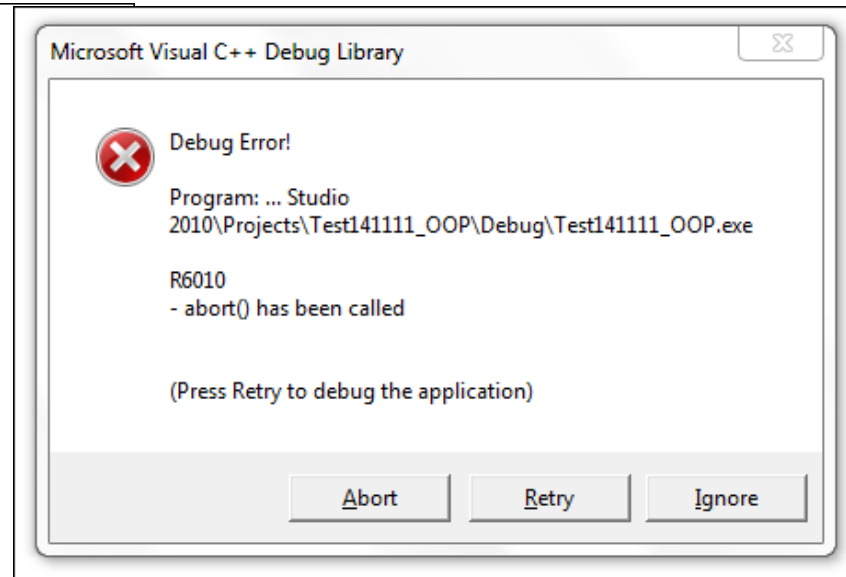
Ví dụ

```
class A
{
public:
    ~A()
    {
        throw 3.14;
    }
};

void test()
{
    A a;
    throw 1;
}
```

Ví dụ - trường hợp 2

```
void main()
{
    try
    {
        test();
    }
    catch(int i)
    {
        cout<<"Vua throw tu test"<<endl;
    }
    catch(double x)
    {
        cout<<"Vua throw tu ham huy cua A"<<endl;
    }
    catch(...)
    {
        cout<<"Tat cac loi deu phai vay day"<<endl;
    }
}
```



Mẫu composite

Bài 9.1 – Mô tả

- ❑ Máy được tạo thành bằng cách lắp ráp các chi tiết lại theo một thứ tự nào đó. Mỗi chi tiết máy có thể là chi tiết đơn hoặc chi tiết phức
- ❑ Chi tiết đơn: là chi tiết không chứa bên trong nó chi tiết nào khác. Thông tin của một chi tiết đơn bao gồm: mã số chi tiết, giá tiền của chi tiết đó

Bài 9.1 – Mô tả

- ❑ Chi tiết phức: là chi tiết chứa bên trong nó nhiều chi tiết con khác nhau. Mỗi chi tiết con như vậy có thể là chi tiết đơn hoặc chi tiết phức.
- ❑ Thông tin của chi tiết phức bao gồm: mã số chi tiết, số lượng chi tiết con và danh sách các chi tiết con. Giá tiền của chi tiết phức bằng tổng giá tiền của các chi tiết con.

Bài 9.1 – Yêu cầu

- ❑ Nhập vào thông tin của một máy
- ❑ Tính tiền của một máy
 - Tiền máy = tổng tiền của các chi tiết trong máy * 200%
- ❑ Xuất các chi tiết trong một máy
- ❑ Nhập vào mã số, tìm chi tiết máy có mã số tương ứng
- ❑ Đếm số lượng chi tiết đơn có trong một máy

Bài 9.2 – Đề bài

- ❑ Mỗi ổ đĩa trên máy tính sẽ chứa nhiều tập tin và thư mục. Thông tin của mỗi ổ đĩa gồm tên ổ đĩa, hãng sản xuất.
- ❑ Mỗi thư mục như vậy lại chứa các thư mục con hoặc các tập tin. Dung lượng của một thư mục bằng dung lượng của các thư mục con và các tập tin trong nó. Thông tin của mỗi thư mục gồm tên thư mục, ẩn(Y/N), chỉ đọc (Y/N).
- ❑ Thông tin của mỗi tập tin gồm tên tập tin, phần mở rộng, dung lượng tập tin, ẩn(Y/N), chỉ đọc (Y/N).

Bài 9.2 – Yêu cầu

- ☐ Nhập vào thông tin của một ổ đĩa.
- ☐ Tính tổng dung lượng của ổ đĩa.
- ☐ Nhập vào một chuỗi. Tìm thư mục/tập tin có tên chứa chuỗi đó.

